

DesignKata

LLD practice with feedback you can check.

Pick a low-level design problem, write short notes and a class diagram, and submit. Rules check the structure in about a second. An AI reviewer then judges six qualities, and every judgment has to quote your own words. Code verifies each quote. Then take the twist: a new requirement, the way an interviewer adds one. DesignKata diffs your two class diagrams and tells you how many existing classes the change forced you to edit.

Built by Devank Srivastava for the CipherSchools engineering assignment, September 2026.

The screenshot shows the DesignKata web application. At the top, there's a header with the DesignKata logo, navigation links for 'Problems' and 'Progress', and a status indicator 'AI review on: deepseek/deepseek-v4.1-flash'. Below the header, the main content area is titled 'PARKING LOT · VERSION 2 · TWIST: EV CHARGING SPOTS'. The central heading is 'Feedback on version 2'. To the right of this heading is a progress bar with three stages: 'SUBMITTED', 'EVALUATING', and 'COMPLETED'. Below the heading, there's a section titled 'WORK ON THIS NEXT' with the sub-heading 'Behaviour and edge cases'. It contains a text prompt: 'Add a short flow for the lost-ticket case showing who looks up the plate and who supplies the daily maximum, and describe how EV charging is settled at exit.' Below this, a paragraph of feedback is shown: 'The design is strong on responsibilities, coupling, and extensibility, with a clean additive twist. Requirements and behaviour are solid but could be more explicit about priorities and the lost-ticket and EV charging flows. Trade-offs are justified but could weigh more alternatives.' Further down is a 'Rubric' section. It contains two columns of criteria. The left column is 'Requirements and scope' (Level 3 of 4) and the right column is 'Responsibilities and cohesion' (Level 4 of 4). Each column has a 'WHAT WORKS' section and a 'CONCERN' section. The 'WHAT WORKS' sections describe what the design does well at, while the 'CONCERN' sections point out areas for improvement. At the top right of the rubric section, it says '17 of 17 quotes found word for word in your design'.

Documents: [Research note](#) · [Design note](#) · [Decision log](#) · [AI usage](#) · PDFs in [docs/pdf/](#)

Try it in 60 seconds

1. Run it locally with the two commands below, or open the live demo if one is linked on the repository page. The free host sleeps when idle, so the first load can take about a minute.
2. Open **Parking Lot**, click **Load sample design**, then **Submit for review**. The sample is a deliberately typical first attempt.
3. Rule findings appear within about a second. The rubric review follows in about 15 seconds. Look for the green "Found in your design" quotes.
4. Click **Take the twist**, add an EV charging spot to the diagram, submit, and read the blast radius.
5. Open **Progress** to see the weaknesses that keep coming back.

Run it locally

Requires **Node 24** (see `.nvmrc`). Nothing else to install.

```
npm install
npm run dev
```

Open <http://localhost:3000>.

- **No database needed.** Without `MONGO_URI`, a local MongoDB starts by itself and keeps its data in `.data/mongo`, so your history survives restarts.
- **No key needed.** Without `OPENROUTER_API_KEY`, feedback is rules-only and the app says so. For the AI review, copy `.env.example` to `.env` and set `OPENROUTER_API_KEY`. The model defaults to `deepseek/deepseek-v4.1-flash`; set `LLM_MODEL` to use another one. `npm run check:llm` confirms the key works.

For production mode, run `npm run build && npm start`. Deployment is described in `render.yaml`: one web service, with MongoDB Atlas as the database.

How it works

The screenshot shows the DesignKata web application. The top navigation bar includes the DesignKata logo, 'Problems', and 'Progress' tabs. A status bar on the right indicates 'AI review on: deepseek/deepseek-v4.1-flash'. The main content area is titled 'Parking Lot' and is labeled 'EASY 45 MINUTE PRACTICE'. It contains a problem description, requirements, and a Mermaid class diagram. The class diagram is written as text in a text area, and a visual representation of the diagram is shown below it. The diagram shows classes like ParkingLot, EntryGate, ExitGate, and Ticket, with their attributes and methods. The interface also includes a 'Revising version 2.' section and a 'Design notes' tab.

DesignKata Problems Progress

AI review on: deepseek/deepseek-v4.1-flash

EASY 45 MINUTE PRACTICE

Parking Lot

A mall in Noida is replacing paper parking slips with an automated lot. Bikes, cars and trucks enter through several gates, get a ticket, park on one of several levels and pay at the exit. The operator wants to change pricing without calling a developer.

Requirements

1. The lot has several levels, and each level has spots of different sizes: compact, regular and large.
2. At entry, a vehicle (bike, car or truck) gets the nearest free spot that fits it, and a ticket with the entry time and the spot.
3. At exit, the fee is calculated from the ticket, paid by cash or UPI, and the spot is freed.
4. A display board shows the free spots per level and per spot size.
5. Pricing changes over time: hourly slabs today, weekend rates or a flat event price later.
6. Several gates can issue tickets at the same moment.

Out of scope

- User accounts and advance booking
- Payment gateway integration
- Sensors, barriers and other hardware

Revising version 2.

Your draft starts from what you last submitted. Submitting creates version 3.

Design notes

Class diagram

Mermaid class diagram

Write the diagram as text. It is parsed, so the rules can check it and diff it.

```
classDiagram
class ParkingLot {
-List~Level~ levels
+entryGate(int id) EntryGate
+exitGate(int id) ExitGate
}
class EntryGate {
-SpotAllocator allocator
+admit(Vehicle v) Ticket
}
class ExitGate {
-PricingStrategy pricing
-PaymentMethod payment
+release(Ticket t) Receipt
}
```

Choose a problem → design (notes + Mermaid diagram) → submit (202, stored first)
→ rules (about 1 s) → AI rubric review, every quote verified (about 15 s)
→ revise (level changes per criterion) or take the twist (blast radius) → progress

A submission is five guided notes (requirements, entities, flows, trade-offs, edge cases) plus a class diagram written as Mermaid text. Because the diagram is text, it can be parsed into classes and relationships, checked, and diffed between versions.

Ten deterministic rules run instantly and cost nothing. They check the required sections, the diagram syntax, and whether the core concepts appear (matched by the names learners actually use). They also flag a god class, a part of the problem that varies but has no abstraction, dependency loops, orphan classes, public state, deep inheritance, and edge cases the notes never mention.

The rubric has six criteria, each on four levels that are described in words, and no total score. The AI reviewer must quote evidence before it picks a level, and `EvidenceVerifier` checks every quote against what you wrote.

When a review is slow or fails, nothing is lost. The submission is stored first and a background worker claims the review with a lease, so rule findings show while the AI works. Provider errors are retried with backoff, and a failed review keeps its rule findings and can be retried. If the worker crashes, its job is picked up again, and the same `Idempotency-Key` never creates a duplicate.

The design note has the domain model, the state machine, the failure table and both change tests.

CONCERN

The design does not describe how the lost-ticket flow checks the plate or how the daily maximum is obtained, and the EV charging flow at exit is not detailed.

NEXT STEP

Add a short flow for the lost-ticket case showing who looks up the plate and who supplies the daily maximum, and describe how EV charging is settled at exit.

Found in your design:

Entry: EntryGate asks SpotAllocator for a spot; the allocator reserves it with a compare-and-set so two gates cannot take the same spot; a Ticket is issued.

Found in your design:

Exit: ExitGate asks PricingStrategy for the fee, charges it through PaymentMethod, and only then releases the spot, which notifies DisplayBoard.

Found in your design:

Lot full: SpotAllocator returns nothing and the gate refuses entry.

The trade-ons section does not weigh alternatives for allocation or payment, and does not discuss the cost of the observer choice or the atomic reservation approach.

NEXT STEP

Add one or two sentences weighing an alternative for allocation (for example, lowest level first versus nearest) and for payment, and note the trade-off of the atomic reservation approach.

Found in your design:

Strategy for pricing, allocation and payment, because each varies independently.

Found in your design:

Observer for the display board so spots do not know about screens.

Found in your design:

Rejected: vehicle subclasses, because behaviour does not differ by type yet; an enum is enough.

How your design absorbed the twist

Measured by diffing the two class diagrams

0

BLAST RADIUS

Additive: no existing class had to change.

Added: EVChargingSpot, KwhPricing

Structure checks

Deterministic, instant and free

No structural problems found.

Revise this design

See your progress

Measured results

What	Result
Calibration: weak, medium and strong designs, 3 reviews each (<code>npm run calibrate</code>)	Ordered correctly on 6 of 6 criteria; the widest spread was 1 level; 119 of 124 quotes verified
Consistency: 5 reviews of one design, in 2 configurations (<code>npm run experiment:consistency</code>)	5 of 6 criteria gave the same level every run; 112 of 115 quotes verified
Dogfood: DesignKata's own class diagram through its own rules (<code>npm run dogfood</code>)	23 classes, no findings

Scripts

Command	What it does
<code>npm run dev</code>	API, worker and React app on one port, with hot reload
<code>npm test</code>	171 tests: unit, integration and end to end, against a real MongoDB with a fake LLM
<code>npm run typecheck</code>	Type-checks the server and the web app
<code>npm run check:text</code>	Fails on em dashes, en dashes or anything that looks like an API key (runs in CI)
<code>npm run build / npm start</code>	Production build and server
<code>npm run check:llm</code>	Confirms the OpenRouter key and model work
<code>npm run calibrate</code>	Checks that the reviewer ranks designs of known quality correctly
<code>npm run experiment:consistency</code>	The research note's experiment
<code>npm run dogfood</code>	Runs DesignKata's own design through its rules
<code>npm run pdf</code>	Builds the submission PDFs with headless Chrome

Project layout

shared/	the contract both sides share: types, and the rubric
server/src/domain/	Attempt, Submission, Evaluation (state machine), DesignModel, Problem
server/src/analysis/	one reader per submission format; the change-impact diff
server/src/evaluation/	the Evaluator seam, rules, LLM rubric evaluator, evidence verifier, pipeline
server/src/llm/	LlmClient seam, OpenRouter client with retries, fake client, structured output
server/src/persistence/	MongoDB models and repositories (idempotency, compare-and-set, leases)
server/src/app/	PracticeService (use cases), EvaluationWorker, progress, rate limiter
server/src/http/	routes, validation and error mapping
server/src/problems/	the four problems, as data
server/test/	unit and integration tests
web/src/	React app: Problems, Workspace, Feedback, Progress
docs/	notes, decision log, diagrams, experiment output, PDFs

Limitations

- No accounts. The browser keeps a random learner id; a real product would use CipherSchools sign-in.
- The rules are heuristics. Thresholds such as eight methods for a god class are starting points to tune with real learners.
- The Mermaid parser covers the syntax learners write, not all of Mermaid.
- The calibration set is small (three designs on one problem), and the reviewer is strict, so level 4 is rare.
- Rate limiting (12 reviews per learner per hour) is per process. A shared store would take over with more than one instance.

Author

Devank Srivastava · [LinkedIn](#) · [GitHub](#)

Licensed under MIT.

AI usage

I built DesignKata with **Claude Code** (Anthropic's coding agent) as my pair programmer. Inside the product, an LLM (**DeepSeek v4.1 Flash via OpenRouter**) is the rubric reviewer.

Who did what:

- **Claude:** wrote most of the code and the first drafts of these documents.
- **Me:** set the constraints, made the calls below, reviewed the work and used the app end to end.
- **The tests:** 171 of them, run in CI, decided whether any of it was actually right.

These are the five decisions where AI help mattered most, and what I did with each.

1. What a learner submits: accepted

- **Claude proposed:** short guided notes plus a class diagram written as Mermaid text, instead of a drawing canvas or full code.
 - **I accepted** because the diagram being text is what makes everything else possible. It can be parsed for deterministic checks and diffed between versions for the blast radius. A canvas would have given me pixels, not structure, and taken days to build.
-

2. Primary research: a poll rejected, an experiment that proved the AI wrong

- **Claude suggested:** a short poll in my college groups.
 - **I said no** and kept to desk research.
 - **What we did instead:** we agreed to measure LLM feedback directly. Claude's hypothesis was that LLM scores on a design are inconsistent.
 - **The data disagreed.** At temperature 0 a bare "rate it 1 to 10" score was perfectly repeatable. It did move from 5 to 6 when one setting changed, and it never said what to fix.
 - **What changed:** the product's thesis moved from "consistent feedback" to "specific, checkable feedback". That is why every judgment has to quote the learner. I'd rather have the thesis corrected by a measurement than keep one that only sounds right.
-

3. The model provider: I overrode the options

- **Claude offered:** Gemini's free tier or a Claude key.
 - **I chose:** OpenRouter with `deepseek/deepseek-v4.1-flash`. I had the key, and it keeps the provider one environment variable away.
 - **Then we measured reasoning on against off.** The six levels were the same, in 15 seconds instead of 41. A learner is waiting for this, so reasoning is off by default.
-

4. Trusting the reviewer: evidence first, verified by code

- **Claude proposed two things:**
 - order the JSON answer so the model quotes evidence before it picks a level;
 - add a verifier that checks every quote against the learner's own text.
 - **I accepted both**, and the verifier proved itself in the browser. It flagged a quote that the model had lifted from *our own rule findings* and presented as the learner's words.
 - **The fix:** we tightened the prompt to quote only from the submission, and kept the verifier. My takeaway: an LLM's evidence is a claim until code checks it.
-

5. AI-written concurrency code needed a skeptic: a bug caught by tests

- **The bug:** Claude's first version of the evaluation worker could report itself idle while a pass was still claiming a job.
- **How it showed up:** every test passed on its own, but one failed now and then in the full suite.
- **The fix:** we did not loosen the test. We fixed the root cause in the worker, so `drain()` now also waits for a running pass.

- **My takeaway:** AI makes code fast, and tests plus review decide whether it is correct. The same review caught smaller things too, such as the model writing "None" as a strength, which the UI now hides.

One more: my own rule, enforced by a machine

I set one rule for the whole project: **no em dashes anywhere**. `npm run check:text` enforces it in CI, and the reviewer's output is cleaned before learners see it.

It earned its place twice. The AI's file-writing tool turned escaped code points into real em dashes inside source files, and the check caught both times.